23.2 进程管理

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~130k tokens，输出 ~5k tokens（本节）

上一节我们分析了 exec 工具如何创建和执行进程。当一个命令因 `yieldMs` 到期或 `background: true` 而转入后台时，Agent 需要一种手段来监控、交互和终止这些进程。这就是**process 工具**和**进程注册表**的职责。

23.2.1 进程注册表

内存中的双 Map 架构

进程注册表位于 `bash-process-registry.ts`，使用两个 `Map` 来管理进程生命周期：

```
// src/agents/bash-process-registry.ts — 核心数据结构
const runningSessions  = new Map<string, ProcessSession>();   // 运行中
const finishedSessions = new Map<string, FinishedSession>();  // 已结束
```

`ProcessSession` 是运行中进程的完整状态：

```typescript
interface ProcessSession {
  id: string;                    // 会话 ID（短码，如 "a3f7bc"）
  command: string;               // 原始命令
  child?: ChildProcessWithoutNullStreams;  // Node.js 子进程句柄
  stdin?: SessionStdin;          // stdin 写入适配器（支持 PTY）
  pid?: number;                  // 操作系统 PID
  startedAt: number;             // 启动时间戳
  cwd?: string;                  // 工作目录

  // 输出管理
  maxOutputChars: number;        // 最大输出字符数（默认 200,000）
  totalOutputChars: number;      // 已产出总字符数
  pendingStdout: string[];       // 待消费的 stdout 缓冲区
  pendingStderr: string[];       // 待消费的 stderr 缓冲区
  aggregated: string;            // 全量输出（有上限截断）
  tail: string;                  // 最后 2000 字符
  truncated: boolean;            // 是否因超出上限而截断

  // 状态
  exited: boolean;
  exitCode?: number | null;
  exitSignal?: NodeJS.Signals | number | null;
  backgrounded: boolean;         // 是否已转入后台
}
```

FinishedSession 是进程结束后的精简记录：

```typescript
interface FinishedSession {
  id: string;
  command: string;
  startedAt: number;
  endedAt: number;
  status: ProcessStatus;         // "completed" | "failed" | "killed"
  exitCode?: number | null;
  exitSignal?: NodeJS.Signals | number | null;
  aggregated: string;            // 保留完整输出
  tail: string;                  // 最后 2000 字符
  truncated: boolean;
  totalOutputChars: number;
}
```

输出缓冲策略

当进程产出输出时， `appendOutput()` 函数负责管理多级缓冲：

```
// src/agents/bash-process-registry.ts — appendOutput（简化）
export function appendOutput(
  session: ProcessSession,
  stream: "stdout" | "stderr",
  chunk: string
) {
  const buffer = stream === "stdout"
    ? session.pendingStdout
    : session.pendingStderr;

  buffer.push(chunk);

  // 1. 如果 pending 缓冲超出上限，截断旧数据
  if (pendingChars > pendingCap) {
    session.truncated = true;
    capPendingBuffer(buffer, pendingChars, pendingCap);
  }

  // 2. aggregated 也有上限（默认 200,000 字符）
  session.aggregated = trimWithCap(
    session.aggregated + chunk,
    session.maxOutputChars
  );

  // 3. tail 始终保留最后 2000 字符
  session.tail = tail(session.aggregated, 2000);
}
```

这个设计解决了实际问题：

- **pending 缓冲**是 `poll` 操作的数据源——Agent 每次 poll 时取走 pending，然后清空
- **aggregated** 是完整输出的滚动窗口——超出上限时丢弃开头的内容
- **tail** 用于快速预览——列出进程时显示最后一点输出

> **衍生解释**：这里的"滚动窗口"概念类似于日志系统中的 Ring Buffer（环形缓冲区）。当新数据写入且缓冲已满时，最旧的数据被丢弃。 `trimWithCap()` 函数的实现很直接： `text.slice(text.length - max)` ，即只保留最后 `max` 个字符。

进程生命周期转换

```
创建                    后台化                  退出
addSession()  →  markBackgrounded()  →  markExited()
   │                    │                    │
   ▼                    ▼                    ▼
runningSessions    仍在 running          moveToFinished()
                   backgrounded=true      ├─ 从 runningSessions 删除
                                          └─ 加入 finished Sessions
```

`moveToFinished()` 有一个关键检查：**只有 backgrounded 的进程才会被保存到 finishedSessions**。如果进程是同步完成的（即在 yieldMs 内返回），它的结果直接返回给 Agent，不需要注册到 finished 表。

自动清理（Sweeper）

```
// src/agents/bash-process-registry.ts — 定时清理
const DEFAULT_JOB_TTL_MS = 30 * 60 * 1000;  // 30 分钟
const MIN_JOB_TTL_MS     = 60 * 1000;        // 最短 1 分钟
const MAX_JOB_TTL_MS     = 3 * 60 * 60 * 1000; // 最长 3 小时

function pruneFinishedSessions() {
  const cutoff = Date.now() - jobTtlMs;
  for (const [id, session] of finishedSessions.entries()) {
    if (session.endedAt < cutoff) {
      finishedSessions.delete(id);
    }
  }
}

function startSweeper() {
  sweeper = setInterval(pruneFinishedSessions,
    Math.max(30_000, jobTtlMs / 6));
  sweeper.unref?.();  // 不阻止 Node.js 进程退出
}
```

sweeper 的间隔为 TTL 的 1/6（最少 30 秒）。使用 `unref()` 确保定时器不会阻止 Node.js 进程的正常退出。

23.2.2 process 工具

工具定义

```
// src/agents/bash-tools.process.ts — Schema
const processSchema = Type.Object({
  action:    Type.String(),                      // 动作类型
  sessionId: Type.Optional(Type.String()),       // 目标会话 ID
  data:      Type.Optional(Type.String()),       // write 的数据
  keys:      Type.Optional(Type.Array(Type.String())),  // send-keys 的按键
  hex:       Type.Optional(Type.Array(Type.String())),  // send-keys 的十六进制字节
  literal:   Type.Optional(Type.String()),       // send-keys 的字面字符串
  text:      Type.Optional(Type.String()),       // paste 的文本
  bracketed: Type.Optional(Type.Boolean()),      // 是否使用 bracketed paste
  eof:       Type.Optional(Type.Boolean()),      // write 后是否关闭 stdin
  offset:    Type.Optional(Type.Number()),       // log 的偏移量
  limit:     Type.Optional(Type.Number()),       // log 的行数限制
});
```

十种动作

process 工具支持十种动作，覆盖进程管理的完整需求：

动作	用途	需要 sessionId
`list`	列出所有后台会话（运行中 + 已完成）	✗
`poll`	获取增量输出（消费 pending 缓冲）	✓
`log`	读取完整日志（支持分页）	✓
`write`	向 stdin 写入数据	✓
`send-keys`	发送终端按键序列	✓
`submit`	发送回车键（ `\r` ）	✓
`paste`	粘贴文本（支持 bracketed paste）	✓
`kill`	终止进程	✓
`clear`	清除已完成会话记录	✓

动作	用途	需要 sessionId
`remove`	强制移除会话（运行中则先杀死）	✓

`list` — 会话列表

```
// src/agents/bash-tools.process.ts — list 动作（简化）
case "list": {
  const running = listRunningSessions()
    .filter(s => isInScope(s))
    .map(s => ({
      sessionId: s.id,
      status: "running",
      pid: s.pid,
      runtimeMs: Date.now() - s.startedAt,
      command: s.command,
      name: deriveSessionName(s.command),
      tail: s.tail,
    }));
  const finished = listFinishedSessions()
    .filter(s => isInScope(s))
    .map(s => ({
      sessionId: s.id,
      status: s.status,
      runtimeMs: s.endedAt - s.startedAt,
      name: deriveSessionName(s.command),
      exitCode: s.exitCode,
    }));
  // 按启动时间倒序排列
  const lines = [...running, ...finished]
    .toSorted((a, b) => b.startedAt - a.startedAt)
    .map(s => `${s.sessionId} ${pad(s.status, 9)} ${formatDuration(s.runtimeMs)} :: ${l
  return { content: [{ type: "text", text: lines.join("\n") }] };
}
```

`deriveSessionName()` 从命令中提取简短名称——取第一个 token 作为"动词"，第一个非选项参数作为"目标"：

```
// src/agents/bash-tools.shared.ts — 名称提取
function deriveSessionName(command: string): string | undefined {
  const tokens = tokenizeCommand(command);
  const verb = tokens[0];
  let target = tokens.slice(1).find(t => !t.startsWith("-"));
  return target ? `${verb} ${truncateMiddle(target, 48)}` : verb;
}
// "npm install express" → "npm express"
// "git log --oneline" → "git log"
```

poll — 增量输出

poll 与 log 的核心区别：

```
poll:
  1. 获取 pendingStdout + pendingStderr
  2. 清空 pending 缓冲（drainSession）
  3. 返回自上次 poll 以来的新输出

log:
  1. 读取 aggregated（全量输出）
  2. 支持 offset/limit 分页
  3. 不修改任何状态
```

poll 是**消费性**的——每次调用后 pending 被清空；log 是**只读性**的——可以反复查看。

write 与 send-keys 的区别

两者都向进程的 stdin 写入数据，但用途不同：

```
// write — 直接写入原始数据
case "write": {
  const stdin = scopedSession.stdin ?? scopedSession.child?.stdin;
  await new Promise((resolve, reject) => {
    stdin.write(params.data ?? "", (err) => err ? reject(err) : resolve());
  });
  if (params.eof) stdin.end();  // 可选关闭 stdin
}

// send-keys — 编码终端按键序列
case "send-keys": {
  const { data, warnings } = encodeKeySequence({
    keys: params.keys,       // ["C-c", "Enter", "M-x"]
    hex: params.hex,         // ["0x1b", "0x5b"]
    literal: params.literal, // "hello"
  });
  await stdin.write(data);
}
```

`write` 适合管道式输入（向脚本传递数据）； `send-keys` 适合与 TUI 程序交互（如在 vim 中按 `Esc` 、 `Ctrl-C` ）。

23.2.3 PTY 按键编码

encodeKeySequence

PTY 按键编码引擎位于 `pty-keys.ts` ， 支持多种输入格式：

```ts
// src/agents/pty-keys.ts — encodeKeySequence（简化）
export function encodeKeySequence(request: KeyEncodingRequest) {
  let data = "";

  // 1. literal — 字面文本直接附加
  if (request.literal) data += request.literal;

  // 2. hex — 十六进制字节序列
  for (const raw of request.hex ?? []) {
    const byte = parseHexByte(raw);  // "0x1b" → 27
    data += String.fromCharCode(byte);
  }

  // 3. keys — 按键 token 编码
  for (const token of request.keys ?? []) {
    data += encodeKeyToken(token, warnings);
  }

  return { data, warnings };
}
```

按键 Token 语法

按键 token 使用类似 tmux/Emacs 的修饰符语法：

Token	含义	编码结果
`Enter`	回车	`\r`
`Tab`	制表符	`\t`
`Escape` / `Esc`	ESC 键	`\x1b`
`BSpace`	退格	`\x7f`
`Up` / `Down` / `Left` / `Right`	方向键	`\x1b[A` ~ `\x1b[D`
`F1` ~ `F12`	功能键	VT100 序列
`C-c`	Ctrl-C	`\x03`
`M-x`	Alt-X	`\x1b` + `x`
`S-Tab`	Shift-Tab	`\x1b[Z`

Token	含义	编码结果
`C-M-a`	Ctrl-Alt-A	`\x1b + \x01`

> **衍生解释**：在 VT100/xterm 终端标准中，特殊按键通过 **转义序列（Escape Sequence）**编码。它们以 ESC 字符（ `\x1b` ）开头，后跟 CSI（Control Sequence Introducer，即 `[` ），再跟参数和终止字符。例如方向键 Up 编码为 `\x1b[A` ，Delete 键编码为 `\x1b[3~` 。这些序列是终端仿真器的标准协议。

修饰符前缀的处理：

```
// src/agents/pty-keys.ts — 修饰符解析
function parseModifiers(token: string) {
  const mods = { ctrl: false, alt: false, shift: false };
  let rest = token;
  while (rest.length > 2 && rest[1] === "-") {
    const mod = rest[0].toLowerCase();
    if (mod === "c") mods.ctrl = true;   // C- = Ctrl
    else if (mod === "m") mods.alt = true;   // M- = Alt/Meta
    else if (mod === "s") mods.shift = true; // S- = Shift
    else break;
    rest = rest.slice(2);
  }
  return { mods, base: rest };
}
```

Ctrl 字符编码

Ctrl 组合键的编码遵循 ASCII 控制字符规则：

```
function toCtrlChar(char: string): string | null {
  if (char === "?") return "\x7f";  // Ctrl-? = DEL
  const code = char.toUpperCase().charCodeAt(0);
  if (code >= 64 && code <= 95) {
    return String.fromCharCode(code & 0x1f);  // 位掩码
  }
  return null;
}
// 'C' (67) & 0x1f = 3 → Ctrl-C = \x03
// 'A' (65) & 0x1f = 1 → Ctrl-A = \x01
```

> **衍生解释**：ASCII 表中，大写字母 A-Z 的编码是 65-90。对这些编码执行 `& 0x1f` （与 31 做按位与），结果恰好是 1-26，对应 ASCII 控制字符 SOH 到 SUB。这就是为什么 Ctrl-C 是 `\x03` （3 = 67 & 31）——这个规则来自 1960 年代的终端硬件设计，并沿用至今。

xterm 修饰符编码

对于方向键等带修饰符的组合（如 Shift-Up），使用 xterm 扩展语法：

```
function xtermModifier(mods: Modifiers): number {
  let mod = 1;
  if (mods.shift) mod += 1;   // 2
  if (mods.alt)   mod += 2;   // 3
  if (mods.ctrl)  mod += 4;   // 5
  return mod;
}

// S-Up → \x1b[1;2A  (1 是默认参数, 2 = shift)
// C-Right → \x1b[1;5C  (5 = ctrl)
// C-S-Left → \x1b[1;6D  (6 = ctrl+shift)
```

Bracketed Paste

```
// src/agents/pty-keys.ts
export const BRACKETED_PASTE_START = `\x1b[200~`;
export const BRACKETED_PASTE_END   = `\x1b[201~`;

export function encodePaste(text: string, bracketed = true): string {
  if (!bracketed) return text;
  return `${BRACKETED_PASTE_START}${text}${BRACKETED_PASTE_END}`;
}
```

> **衍生解释**：Bracketed Paste 是现代终端仿真器的一个安全特性。当用户粘贴文本时，终端会在文本前后插入特殊序列 `\x1b[200~` 和 `\x1b[201~` 。Shell（如 zsh、bash 5.1+）检测到这些序列后，知道内容是粘贴的而非用户手动输入的，因此不会立即执行——防止"粘贴攻击"（恶意网页在剪贴板中放入 `rm -rf /` 等命令）。OpenClaw 的 `paste` 动作默认使用 bracketed 模式来模拟正确的粘贴行为。

23.2.4 作用域隔离

process 工具支持 **scopeKey** 来实现进程隔离：

```
// src/agents/bash-tools.process.ts — 作用域过滤
const scopeKey = defaults?.scopeKey;
const isInScope = (session) =>
  !scopeKey || session?.scopeKey === scopeKey;
```

当多个 Agent 实例并行运行时，每个 Agent 只能看到和操作自己 scopeKey 下的进程。这防止了 Agent A 误杀 Agent B 的后台任务。

本节小结

1. **进程注册表使用双 Map 架构**： `runningSessions` （运行中）和 `finishedSessions` （已结束），通过 `markExited()` 在两者之间转移
2. **三级输出缓冲**：pending（增量消费）、aggregated（滚动窗口，默认 200K 字符上限）、tail（最后 2000 字符预览）
3. **process 工具支持十种动作**：list、poll、log、write、send-keys、submit、paste、kill、clear、remove
4. **PTY 按键编码支持完整的终端按键语法**：修饰符前缀（C-/M-/S-）、命名键（Enter/Tab/F1-F12）、hex 字节、xterm 修饰符扩展
5. **Ctrl 字符编码利用 ASCII 位掩码规则**（ `& 0x1f` ），这是终端硬件时代的设计遗产
6. **Bracketed Paste** 防止粘贴攻击——在粘贴文本两端加入特殊转义序列
7. **自动清理器（sweeper）** 每隔 TTL/6 扫描一次已完成会话，默认 30 分钟后淘汰
8. **scopeKey 隔离**确保多 Agent 并行时互不干扰